

Migraciones en SQL (PostgreSQL)

Diego Muñoz

28 Mayo 2026

¿Qué es una migración?

Registro de cambios estructurales

- Una migración es un conjunto de instrucciones SQL que modifica el **esquema** de una base de datos.
- Se usa para **crear, alterar o eliminar** estructuras como tablas, columnas, restricciones.
- Permite aplicar cambios de forma **controlada, secuencial y reversible**.

¿Por qué usar migraciones?

- Para mantener el esquema **versionado** igual que el código fuente.
- Para asegurar que todos los entornos tengan el mismo diseño de base de datos.
- Para aplicar cambios **de manera segura y trazable**.

Cambios típicos en migraciones

- Crear o eliminar tablas:

```
CREATE TABLE producto (...);
```

```
DROP TABLE producto;
```

- Agregar o quitar columnas:

```
ALTER TABLE producto ADD COLUMN precio DECIMAL;
```

```
ALTER TABLE producto DROP COLUMN precio;
```

- Modificar tipos o restricciones:

```
ALTER TABLE producto ALTER COLUMN nombre TYPE VARCHAR(100);
```

```
ALTER TABLE producto ADD CONSTRAINT nombre_unico UNIQUE(nombre);
```

Renombrar una columna

- En PostgreSQL el rename es atómico y conserva los datos:

```
ALTER TABLE producto RENAME COLUMN nombre TO descripcion;
```

- **Cuidado:** si hay código vivo leyendo nombre, el rename rompe la aplicación en el instante en que se aplica.

Renombrar de forma segura (expand/contract)

Cuando no puedes detener la aplicación, el rename se hace en pasos:

1. `ALTER TABLE producto ADD COLUMN descripcion TEXT;`
2. Backfill: `UPDATE producto SET descripcion = nombre;`
3. Desplegar código que escribe en ambas columnas y lee de la nueva.
4. `ALTER TABLE producto DROP COLUMN nombre;` cuando ya nadie la usa.

Cambiar el tipo de una columna

Conversiones directas:

- `VARCHAR(50) → VARCHAR(200), VARCHAR → TEXT.`
- `SMALLINT → INTEGER → BIGINT.`
- `DECIMAL(10,2) → DECIMAL(12,2).`

```
ALTER TABLE producto ALTER COLUMN nombre TYPE VARCHAR(200);
```

Cambiar el tipo: conversiones no directas

Pierden información o requieren interpretar el dato. Se indica al motor cómo hacerlas con USING:

- VARCHAR(200) → VARCHAR(50).
- DECIMAL → INTEGER.
- TEXT → INTEGER, TEXT → DATE.

```
ALTER TABLE producto
```

```
  ALTER COLUMN precio TYPE INTEGER
```

```
  USING precio::INTEGER;  -- trunca: 12.99 -> 12
```


Cambio de tipo: precauciones

- ALTER ... TYPE toma un **lock exclusivo** y puede **reescribir toda la tabla**. En tablas grandes es lento y bloquea lecturas.
- Si la conversión falla para alguna fila, la migración aborta entera.
- Mejor limpiar los datos antes, o usar una columna intermedia: agregar la nueva, hacer backfill validado, borrar la vieja.

Transformar la forma de los datos

Ejemplo: separar fecha DATE en dia / mes / anio INT. Se hace en **tres migraciones** independientes:

```
-- 00X_add_dia_mes_anio.sql  
ALTER TABLE evento ADD COLUMN dia INT;  
ALTER TABLE evento ADD COLUMN mes INT;  
ALTER TABLE evento ADD COLUMN anio INT;
```

Backfill y limpieza

```
-- 00Y_backfill_dia_mes_anio.sql
```

```
UPDATE evento
```

```
    SET dia  = EXTRACT(DAY   FROM fecha)::INT,
```

```
        mes  = EXTRACT(MONTH FROM fecha)::INT,
```

```
        anio = EXTRACT(YEAR  FROM fecha)::INT;
```

```
-- 00Z_drop_fecha.sql
```

```
ALTER TABLE evento DROP COLUMN fecha;
```

- El caso inverso, unir columnas, es análogo: nombre || ' ' || apellido.
- Pasos separados → cada uno es revertible y observable.

Agregar una columna NOT NULL

ADD COLUMN ... NOT NULL sin DEFAULT falla si la tabla ya tiene filas: no hay valor que poner en las existentes.

-- falla si producto tiene filas

```
ALTER TABLE producto ADD COLUMN stock INTEGER NOT NULL;
```

Se hace en tres pasos:

```
ALTER TABLE producto ADD COLUMN stock INTEGER;
```

```
UPDATE producto SET stock = 0 WHERE stock IS NULL;
```

```
ALTER TABLE producto ALTER COLUMN stock SET NOT NULL;
```

Agregar columna con DEFAULT

```
ALTER TABLE producto ADD COLUMN activo BOOLEAN DEFAULT TRUE;
```

- En PostgreSQL 11 o superior, si el DEFAULT es un valor constante, el motor lo registra como metadato y no toca las filas existentes. Es instantáneo.
- En versiones anteriores, o si el DEFAULT es una función (`now()`, `gen_random_uuid()`), se reescribe toda la tabla.

El costo de la misma sentencia depende del motor y la versión.

Agregar una FOREIGN KEY

Crear el FK valida todas las filas existentes contra la tabla referenciada. En tablas grandes es lento y bloquea escrituras.

```
ALTER TABLE pedido  
  ADD CONSTRAINT fk_cliente  
  FOREIGN KEY (cliente_id) REFERENCES cliente(id);
```

FOREIGN KEY en dos pasos

```
ALTER TABLE pedido
  ADD CONSTRAINT fk_cliente
  FOREIGN KEY (cliente_id) REFERENCES cliente(id)
  NOT VALID;  -- aplica solo a filas nuevas
```

```
ALTER TABLE pedido VALIDATE CONSTRAINT fk_cliente;
```

- NOT VALID agrega la restricción sin revisar lo existente.
- VALIDATE CONSTRAINT recorre las filas viejas después, sin bloquear escrituras.

- Archivos SQL numerados secuencialmente:
 - 001_create_producto.sql
 - 002_add_precio_to_producto.sql
 - 003_create_index_nombre.sql
- Cada archivo contiene **una sola modificación estructural** y puede tener comentarios descriptivos.

Ejemplo 1: crear una tabla

```
-- 001_create_producto.sql  
CREATE TABLE producto (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT NOT NULL  
);
```

Ejemplo 2: agregar una columna

```
-- 002_add_precio_to_producto.sql  
ALTER TABLE producto ADD COLUMN precio DECIMAL(10,2);
```

Ejemplo 3: agregar índice

```
-- 003_create_index_nombre.sql  
CREATE INDEX idx_producto_nombre ON producto(nombre);
```

Tabla de control:

```
CREATE TABLE schema_migrations (  
    version TEXT PRIMARY KEY,  
    applied_at TIMESTAMP DEFAULT now()  
);
```

- Guarda el nombre o número de cada migración aplicada.

Aplicar migraciones manualmente

1. Ejecutar el archivo SQL.
2. Registrar la migración aplicada:

```
INSERT INTO schema_migrations (version) VALUES ('001_create_producto');
```

Migración auto-registrada

El INSERT se incluye dentro del mismo archivo, en una transacción con el cambio de esquema:

```
-- 001_create_producto.sql
```

```
BEGIN;
```

```
CREATE TABLE producto (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT NOT NULL  
);
```

```
INSERT INTO schema_migrations (version) VALUES ('001_create_producto');
```

```
COMMIT;
```

Aplicar solo las pendientes

Antes de ejecutar, se consulta qué versiones ya están aplicadas:

```
SELECT version FROM schema_migrations;
```

Loop:

1. Listar los archivos del directorio en orden.
2. Saltar los que ya están en `schema_migrations`.
3. Ejecutar el resto.

La PRIMARY KEY sobre `version` evita duplicados si alguien reaplica una migración por error.

Ver migraciones aplicadas

```
SELECT * FROM schema_migrations ORDER BY applied_at;
```


Revertir migraciones (rollback manual)

- Crea archivos de reversión opcionales, por ejemplo:

```
-- 002_rollback_add_precio.sql
```

```
ALTER TABLE producto DROP COLUMN precio;
```

- El reverso no siempre es trivial: un DROP COLUMN es **pérdida irreversible** de datos salvo que tengas backup.

- **NO EDITES MIGRACIONES YA APLICADAS.**
- Usa nombres claros y consistentes: `003_add_stock_column.sql`.
- Aplica y registra cada migración **de una en una**.
- Haz backup antes de aplicar migraciones en producción.
- Separa las migraciones destructivas (`DROP COLUMN`, `DROP TABLE`) en su propio archivo y revísalas dos veces.
- Prueba la migración sobre una **copia de producción**, no solo sobre la BD vacía de desarrollo.

- Una migración es un conjunto de instrucciones SQL para modificar el esquema de una base de datos.
- En SQL se aplican directa y manualmente.
- Las migraciones son secuenciales.
- Dependen de una tabla de control manual.
- Existen herramientas para automatizar migraciones.

¿Tienes dudas? ¡Hablemos!